

Implementación de una IA local en una máquina virtual Ubuntu

Taller de Sistemas Operativos · Laboratorio Ubuntu VM + Ollama
Proyecto final: AsistentelA-VM

Integrantes	Angel Arcos Sebastian Coral Juan Orjuela Javier Rosero
Docente	Mg. Iván Darío Méndez Aguilera
Fecha	7 de octubre de 2026
Entorno	VirtualBox 7.2.16 · Ubuntu 24.04.5 LTS · Ollama 0.34.4

Todo lo hicimos dentro de una VM Ubuntu. En cada captura de terminal lo primero que se ve es echo "Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero" y la fecha. El código (la interfaz, el Modelfile y un script por ejercicio) está en la carpeta codigo/ del repositorio.

Introducción: la IA y por qué correrla local

¿Qué es la inteligencia artificial?

En un programa normal, el programador escribe las reglas una por una (si pasa esto, haga aquello). En la inteligencia artificial que se usa hoy, en cambio, el programa aprende las reglas solo a partir de muchos ejemplos. Durante el entrenamiento ajusta millones de números internos (los *pesos*) hasta que sus respuestas se parecen a las de los ejemplos. Después ya no se entrena más; solo se usa, y a eso se le llama *inferencia*.

Los modelos que usamos en el taller son modelos de lenguaje (LLM). Lo único que hacen es predecir cuál es la palabra (el *token*) que sigue, una y otra vez, hasta armar la respuesta. Para eso tienen que recorrer todos sus pesos en cada palabra. Por eso necesitan tanta memoria y tanta CPU, y por eso sirven como carga de trabajo para estudiar el sistema operativo. Ollama es el programa que carga esos modelos y los deja disponibles como un servicio.

IA local frente a IA en la nube

	IA en la nube (ChatGPT, Gemini, Claude)	IA local (lo que hicimos con Ollama)
Dónde corre	En servidores de la empresa, al otro lado de internet	En nuestro propio equipo, en este caso dentro de la VM
Privacidad	Las preguntas salen del equipo y las procesa un tercero	Nada sale de la máquina
Internet	Sin conexión no funciona	Solo se necesita para bajar el modelo; después funciona sin internet
Tamaño y calidad	Modelos enormes, con respuestas mucho mejores	Modelos pequeños (de 1B a 3B en nuestra VM), que se equivocan más
Velocidad	Por lo general responde en segundos	Depende del equipo: en la VM sin GPU fueron entre 4 y 24 tok/s, y una respuesta podía tardar más de un minuto
Costo	Suscripción o pago por uso	Gratis por consulta; solo se paga el equipo y la luz
Control	Solo lo que permita el proveedor	Control total: modelo, temperatura, contexto, prompt de sistema (lo hicimos con el Modelfile)

En resumen, la nube gana en calidad y velocidad, y la IA local gana en privacidad, en control y en que funciona sin internet. Para un laboratorio de sistemas operativos la local es mejor, porque podemos ver con nuestras propias herramientas qué procesos, memoria, disco y puertos usa.

Cinco modelos viables para nuestro equipo

La VM corre en un portátil con Intel Core i9-12900H, 16 GB de RAM y una NVIDIA RTX 3070 Ti Laptop con 8 GB de VRAM. Pero la VM no puede usar esa GPU, así que adentro solo tiene CPU y 6 GB de RAM. Estos son

los cinco modelos que manejamos, todos de la biblioteca de Ollama:

Modelo	Tamaño	En la VM (solo CPU, 6 GB)	En el portátil con la GPU	Qué hicimos con él
smollm2:135m	270 MB	Sí, el más liviano	Sí	Lo descargamos y borramos en el ejercicio 7
gemma3:1b	815 MB	Sí	Sí	Lo probamos en el ejercicio 8
qwen2.5:1.5b	986 MB	Sí	Sí	Lo probamos en el ejercicio 8
llama3.2:3b	2,0 GB	Sí, aunque más lento	Sí	Es la base de nuestro asistente
qwen2.5:7b	4,7 GB	No: cargado ocuparía unos 5,5 GB de los 6 GB	Sí, cabe en los 8 GB de VRAM	Lo intentamos y la descarga se cortó

La regla que sacamos es sencilla: el modelo, ya cargado, tiene que caber en la memoria con espacio de sobra para el sistema. En la VM eso deja como máximo los modelos de unos 3B. Con la GPU del portátil se podría subir hasta los de 7B u 8B, que responden bastante mejor.

Tipos de IA local y si se pueden usar en nuestro equipo

Tipo	Para qué sirve y ejemplos	En la VM (solo CPU)	En el portátil con la GPU
Modelos de lenguaje	Conversar, explicar, resumir, escribir código (llama3.2, gemma3, qwen2.5)	Sí, es lo que hicimos en el taller	Sí, y mucho más rápido
Embeddings	Convertir texto en números para buscar por significado, por ejemplo para que el asistente consulte documentos propios (nomic-embed-text, 274 MB)	Sí, son modelos pequeños	Sí
Visión	Describir o leer imágenes (gemma3:4b de 3,3 GB, llava:7b de 4,7 GB)	El de 4B cabría justo y sería lento; el de 7B no	Sí
Voz	Pasar audio a texto (Whisper) o texto a voz (Piper)	Sí con las versiones pequeñas, pero lento	Sí
Generación de imágenes	Crear imágenes a partir de texto (Stable Diffusion)	En la práctica no: sin GPU tarda demasiado	Sí, las versiones 1.5 y XL caben en 8 GB de VRAM

En el taller solo probamos modelos de lenguaje. Lo de los otros tipos lo sacamos comparando el tamaño de esos modelos con lo que vimos con los de texto en la VM, no de pruebas propias.

1. Hardware y máquina virtual (secciones 00 y 02)

No quisimos tocar ninguna VM que ya usábamos en el curso, así que clonamos una Ubuntu 24.04 con VBoxManage clonevm y trabajamos sobre la copia. La dejamos con red NAT, como pide la guía. Antes de

instalar Ollama tomamos un snapshot llamado **antes-de-ollama**, por si algo salía mal, y al final otro llamado **taller-completo**.

	Equipo anfitrión	VM Ubuntu-Ollama
Sistema	Windows 11 Home	Ubuntu 24.04.5 LTS, kernel 7.0.0-34-generic
CPU	Intel Core i9-12900H, 14 núcleos / 20 hilos	4 vCPU
RAM	16 GB	6 GB (empezamos con 8 GB, ver abajo) y 3,8 GB de swap
Disco	SSD con ~450 GB libres	Disco virtual de 40 GB (lo ampliamos desde 25 GB)
GPU	NVIDIA RTX 3070 Ti Laptop	Ninguna. VirtualBox no le pasa la GPU a la VM, así que Ollama corre solo con CPU
Red		NAT, con el puerto 22 reenviado a 127.0.0.1:2222 del anfitrión para entrar por SSH

El disco de 25 GB se quedaba corto para los modelos, así que lo ampliamos a 40 GB con `vBoxManage modifymedium --resize 40960` y después, dentro de Ubuntu, con `growpart /dev/sda 2` y `resize2fs /dev/sda2`.

Actividad: ¿qué pasa si se le asigna demasiado a la VM?

Nos pasó de verdad. Al principio le dimos 8 GB de RAM, que es la mitad de lo que tiene el portátil. Con el navegador y otros programas abiertos en Windows, la VM se congeló en la primera consulta al modelo: el reloj de Ubuntu se quedó quieto, el SSH dejó de responder (`Connection timed out during banner exchange`) y ni siquiera el botón de apagado le hacía caso. En ese momento Windows tenía solo 0,86 GB libres, y de los 8 GB "asignados" solo 4,4 GB estaban de verdad en RAM. El resto estaba en disco.

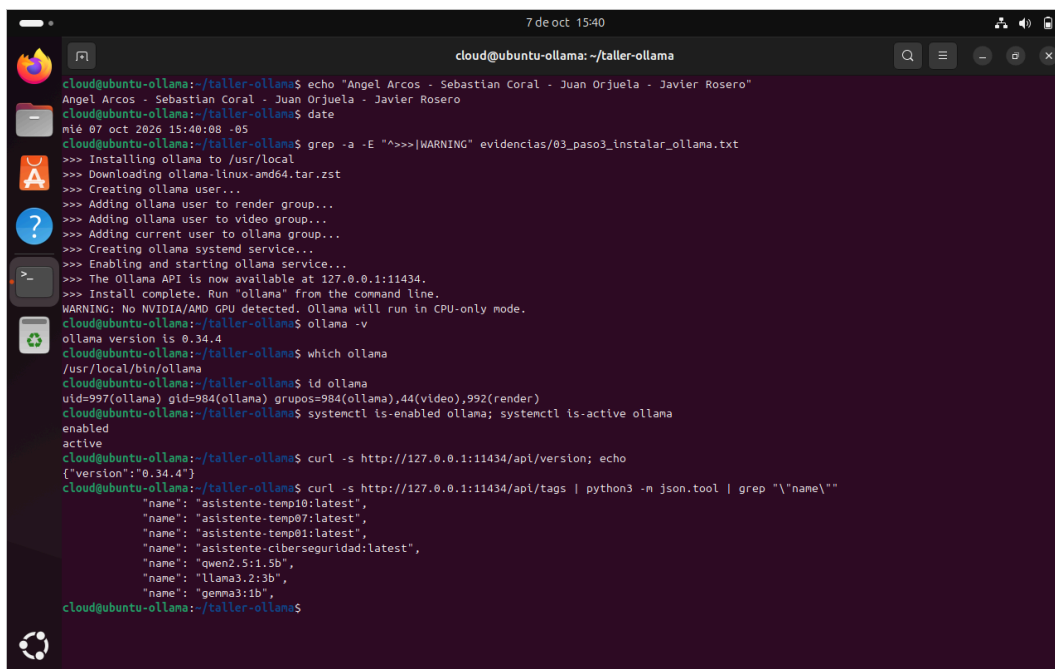
Ahí entendimos que la RAM de una VM no se aparta de verdad: para Windows es memoria normal del proceso de VirtualBox. Si no alcanza, Windows manda parte de esa "RAM" al disco. Ubuntu cree que tiene 8 GB, pero cada vez que toca una de esas páginas hay que traerla del disco, y todo se vuelve lentísimo. Es lo peor que le puede pasar a un modelo de IA, que en cada palabra recorre todos sus pesos. Con las vCPU pasa algo parecido: si se asignan más de las que el anfitrión puede atender, la VM se queda esperando turno. Por eso bajamos la VM a 6 GB.

2. Instalación y verificación de Ollama (sección 03)

Seguimos los pasos de la guía en orden:

1. `sudo apt update` y `sudo apt upgrade -y`. Se actualizó todo, incluido el kernel (pasó de 7.0.0-31 a 7.0.0-34, y quedó activo después de reiniciar).
2. `sudo apt install -y curl wget git pciutils lshw htop net-tools`
3. `curl -fsSL https://ollama.com/install.sh | sh`. El instalador crea un usuario de sistema llamado `ollama`, lo mete en los grupos `render` y `video`, crea el servicio de `systemd` y al final avisa que no encontró GPU y que va a trabajar solo con CPU.
4. `ollama -v` dio la versión 0.34.4 y `which ollama` mostró `/usr/local/bin/ollama`.

5. Del 5 al 7: el servicio quedó enabled y active, y la API respondió en el puerto 11434 (al principio /api/tags devolvía {"models":[]} porque no habíamos bajado ningún modelo).



```
cloud@ubuntu-ollama: ~/taller-ollama
cloud@ubuntu-ollama:~/taller-ollama$ echo "Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero"
Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero
cloud@ubuntu-ollama:~/taller-ollama$ date
mié 07 oct 2026 15:40:08 -05
cloud@ubuntu-ollama:~/taller-ollama$ grep -a -E "^>>>|WARNING" evidencias/03_paso3_instalar_ollama.txt
>>> Installing ollama to /usr/local
>>> Downloading ollama-linux-amd64.tar.zst
>>> Creating ollama user...
>>> Adding ollama user to render group...
>>> Adding ollama user to video group...
>>> Adding current user to ollama group...
>>> Creating ollama systemd service...
>>> Enabling and starting ollama service...
>>> The Ollama API is now available at 127.0.0.1:11434.
>>> Install complete. Run "ollama" from the command line.
WARNING: No NVIDIA/AMD GPU detected. Ollama will run in CPU-only mode.
cloud@ubuntu-ollama:~/taller-ollama$ ollama -v
ollama version is 0.34.4
cloud@ubuntu-ollama:~/taller-ollama$ which ollama
/usr/local/bin/ollama
cloud@ubuntu-ollama:~/taller-ollama$ id ollama
uid=997(ollama) gid=984(ollama) grupos=984(ollama),44(video),992(render)
cloud@ubuntu-ollama:~/taller-ollama$ systemctl is-enabled ollama; systemctl is-active ollama
enabled
active
cloud@ubuntu-ollama:~/taller-ollama$ curl -s http://127.0.0.1:11434/api/version; echo
{"version":"0.34.4"}
cloud@ubuntu-ollama:~/taller-ollama$ curl -s http://127.0.0.1:11434/api/tags | python3 -m json.tool | grep '"name"'
  "name": "asistente-temp10:latest",
  "name": "asistente-temp07:latest",
  "name": "asistente-temp01:latest",
  "name": "asistente.ciberseguridad:latest",
  "name": "qwen2.5:1.5b",
  "name": "llama3.2:3b",
  "name": "gemma3:1b",
```

Captura 1. Mensajes del instalador, versión, ruta del binario, usuario ollama con sus grupos, servicio habilitado y activo, y la API respondiendo.

3. Administración de Ollama (sección 04)

Revisamos el servicio con `systemctl status`, los logs con `journalctl -u ollama`, el puerto con `ss -lntp` y la memoria con `free -h` y `htop`. El comando `nvidia-smi` no existe en la VM, lo cual tiene sentido porque no hay GPU. La medición de RAM en reposo y con el modelo cargado, que pide la actividad de esta sección, está en el ejercicio 3.

AllowedCPUs usa los cgroups del kernel para que Ollama no pueda tocar la CPU 3. Esa queda libre para el sistema, el SSH y el escritorio. Con eso solo no bastó: Ollama seguía lanzando 4 hilos sobre 3 CPU y se volvía muy lento. Por eso también pusimos num_thread 3 en el Modelfile. Con los dos cambios la VM no se volvió a congelar en todo el taller.

La solución de fondo sería apagar Hyper-V en Windows (bcdedit /set hypervisorlaunchtype off), pero no lo hicimos porque eso desactiva otras cosas del equipo, como WSL2.

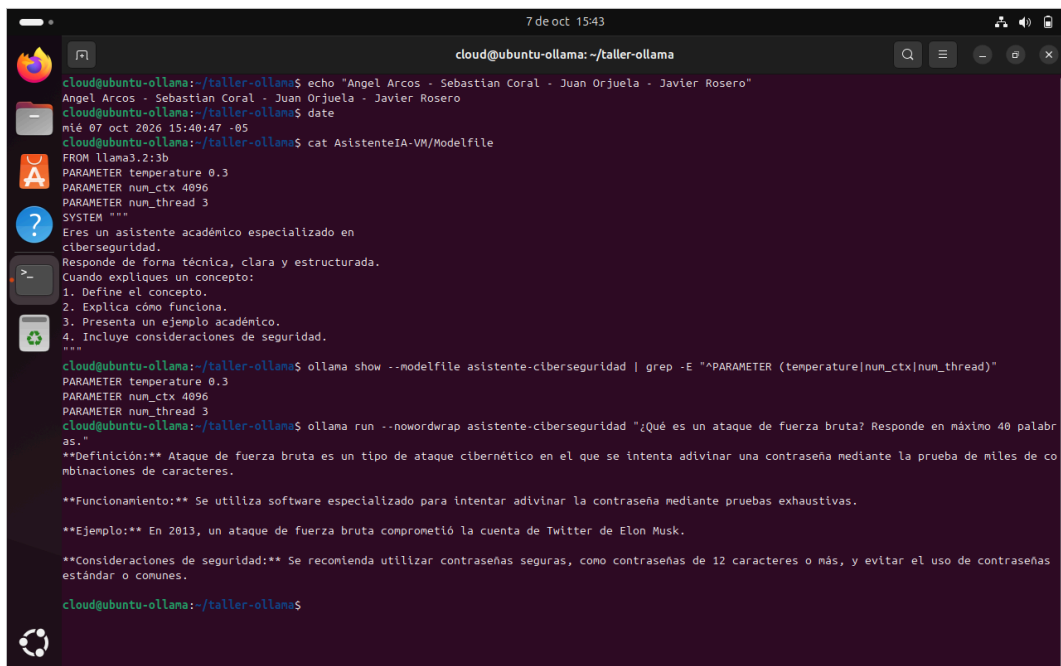
4. Descarga y administración de modelos (sección 05)

Bajamos tres modelos de la biblioteca de Ollama: **gemma3:1b**, **qwen2.5:1.5b** y **llama3.2:3b**. También intentamos con **qwen2.5:7b**, pero no funcionó en la VM. La tabla con tamaño, memoria, velocidad, si funciona y calidad de cada uno está en el ejercicio 8, y el efecto en el disco está en el ejercicio 7.

Usamos todos los comandos de la sección: pull y list (capturas 2 y 11), ps y stop (capturas 7 y 8), show y run (captura 3), y cp y rm (captura 11). Con cp nos dimos cuenta de algo curioso: la copia tiene el mismo ID que el original, porque no duplica el archivo del modelo, solo crea otro nombre que apunta al mismo.

5. Modelo personalizado con Modelfile (sección 06)

Usamos el Modelfile de la guía, con llama3.2:3b como base (en el ejercicio 8 explicamos por qué), y le agregamos num_thread 3 por el problema de la sección 3. Lo creamos con ollama create asistente-ciberseguridad -f Modelfile y revisamos los parámetros con ollama show --modelfile.



```
cloud@ubuntu-ollama: ~/taller-ollama
cloud@ubuntu-ollama$ echo "Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero"
Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero
cloud@ubuntu-ollama:~/taller-ollama$ date
mié 07 oct 2026 15:48:47 -05
cloud@ubuntu-ollama:~/taller-ollama$ cat AsistenteIA-VM/Modelfile
FROM llama3.2:3b
PARAMETER temperature 0.3
PARAMETER num_ctx 4096
PARAMETER num_thread 3
SYSTEM """
Eres un asistente académico especializado en
ciberseguridad.
Responde de forma técnica, clara y estructurada.
Cuando expliques un concepto:
1. Define el concepto.
2. Explica cómo funciona.
3. Presenta un ejemplo académico.
4. Incluye consideraciones de seguridad.
"""
cloud@ubuntu-ollama:~/taller-ollama$ ollama show --modelfile asistente-ciberseguridad | grep -E "(PARAMETER (temperature|num_ctx|num_thread))"
PARAMETER temperature 0.3
PARAMETER num_ctx 4096
PARAMETER num_thread 3
cloud@ubuntu-ollama:~/taller-ollama$ ollama run --nowordwrap asistente-ciberseguridad "¿Qué es un ataque de fuerza bruta? Responde en máximo 40 palabras."
**Definición:** Ataque de fuerza bruta es un tipo de ataque cibernético en el que se intenta adivinar una contraseña mediante la prueba de miles de combinaciones de caracteres.

**Funcionamiento:** Se utiliza software especializado para intentar adivinar la contraseña mediante pruebas exhaustivas.

**Ejemplo:** En 2013, un ataque de fuerza bruta comprometió la cuenta de Twitter de Elon Musk.

**Consideraciones de seguridad:** Se recomienda utilizar contraseñas seguras, como contraseñas de 12 caracteres o más, y evitar el uso de contraseñas estándar o comunes.

cloud@ubuntu-ollama:~/taller-ollama$
```

Captura 3. El Modelfile, los parámetros que confirma ollama show --modelfile y una pregunta con ollama run.

El modelo siempre respeta las cuatro partes que pide el prompt de sistema (definición, cómo funciona, ejemplo y consideraciones de seguridad), pero inventa cosas. En la captura dice que "en 2013 un ataque de

fuerza bruta comprometió la cuenta de Twitter de Elon Musk", y eso no pasó. El caso real que más se le parece es el hackeo de varias cuentas de Twitter en 2020, y fue por ingeniería social, no por fuerza bruta. En otras pruebas inventó un "estudio de 1000 contraseñas" y hasta una cita de una revista (ACM Transactions, 2019). Por eso todo lo que dice hay que revisarlo.

Experimento de temperatura

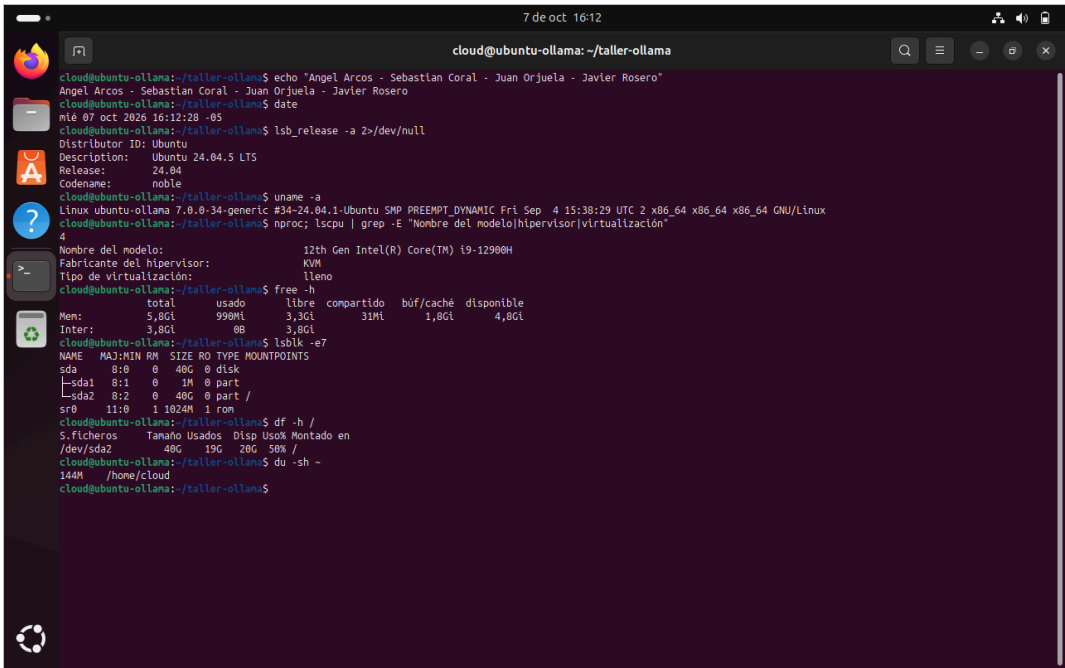
Creamos tres versiones del asistente con temperatura 0.1, 0.7 y 1.0, y a cada una le hicimos dos veces la misma pregunta: "¿Qué es un firewall? Máximo 90 palabras." Luego medimos qué tanto se parecían las dos respuestas de cada versión.

Temperatura	Parecido entre las 2	
	respuestas	Lo que notamos
0.1	50 %	Casi iguales. Repite frases como "capa de seguridad" y "controlador de acceso"
0.7	32 %	Mantiene la estructura, pero cambia la definición y el ejemplo (en una habla de malware y en la otra de VPN)
1.0	29 %	La que más cambia y la menos precisa: pone de ejemplo "la interfaz de usuario de Windows Defender" y escribe "tablas de acceso control"

La temperatura controla qué tan arriesgado es el modelo al escoger la siguiente palabra. Baja, escoge casi siempre la más probable; alta, se arriesga con opciones menos probables. Eso se vio en los resultados. Aun así, con 0.1 las respuestas no salieron idénticas, porque no fijamos una semilla (*seed*) y sigue habiendo azar. En una primera prueba nos dio 40 %, 49 % y 24 %, así que con solo dos respuestas por versión no se puede sacar una conclusión fuerte entre 0.1 y 0.7. Para un asistente técnico nos quedamos con temperatura baja (0.3).

6. Ejercicios prácticos (sección 07)

Ejercicio 1: identificación del sistema operativo



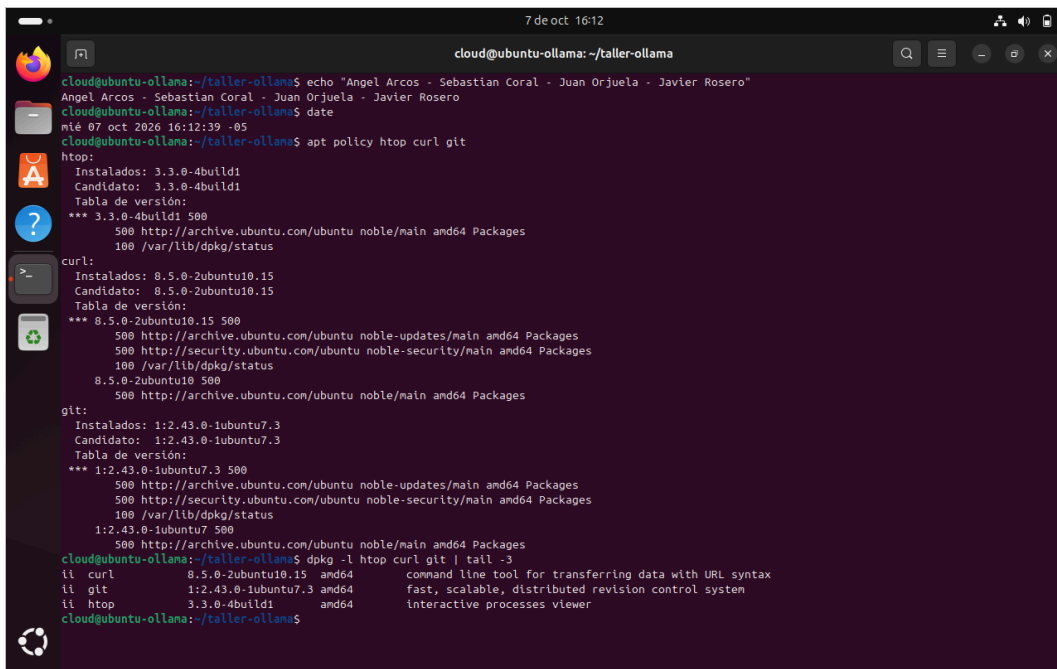
```
cloud@ubuntu-ollama: ~/taller-ollama
cloud@ubuntu-ollama:~/taller-ollama$ echo "Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero"
Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero
cloud@ubuntu-ollama:~/taller-ollama$ date
mié 07 oct 2026 16:12:28 -05
cloud@ubuntu-ollama:~/taller-ollama$ lsb_release -a 2>/dev/null
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.5 LTS
Release:        24.04
Codename:       noble
cloud@ubuntu-ollama:~/taller-ollama$ uname -a
Linux ubuntu-ollama 7.0.0-34-generic #34-24.04.1-Ubuntu SMP PREEMPT_DYNAMIC Fri Sep  4 15:38:29 UTC 2 x86_64 x86_64 GNU/Linux
cloud@ubuntu-ollama:~/taller-ollama$ nproc | lscpu | grep -E "Nombre del modelo|hipervisor|virtualización"
4
Nombre del modelo:      12th Gen Intel(R) Core(TM) i9-12900H
Fabricante del hipervisor: KVM
Tipo de virtualización:  lleno
cloud@ubuntu-ollama:~/taller-ollama$ free -h
              total        used        libre compartido  búf/caché disponible
Mem:          5,8Gi      990Mi      3,3Gi       31Mi       1,8Gi       4,8Gi
Inter:         3,8Gi         0B        3,8Gi
cloud@ubuntu-ollama:~/taller-ollama$ lsblk -e7
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda   8:0    0  40G  0 disk
├─sda1 8:1    0    1M  0 part
├─sda2 8:2    0   40G  0 part /
sr0   11:0   1 1024M  1 rom
cloud@ubuntu-ollama:~/taller-ollama$ df -h /
Filesystem      Tamaño Usados  Disp Usos% Montado en
/dev/sda2        40G   19G   28G   50% /
cloud@ubuntu-ollama:~/taller-ollama$ du -sh ~
144M  /home/cloud
cloud@ubuntu-ollama:~/taller-ollama$
```

Captura 4. lsb_release, uname, lscpu, free, lsblk, df y du en la VM.

Versión	Kernel	Arquitectura	vCPU	RAM	Disco
Ubuntu 24.04.5 LTS (noble)	7.0.0-34-generic	x86_64	4	5,8 GiB	/dev/sda2 de 40 G, 50 % usado

Algo que nos llamó la atención: lscpu dice que el hipervisor es "KVM", aunque estamos en VirtualBox. Es porque VirtualBox le muestra a Ubuntu una interfaz compatible con KVM para que el sistema funcione mejor dentro de la VM.

Ejercicio 2: gestión de paquetes

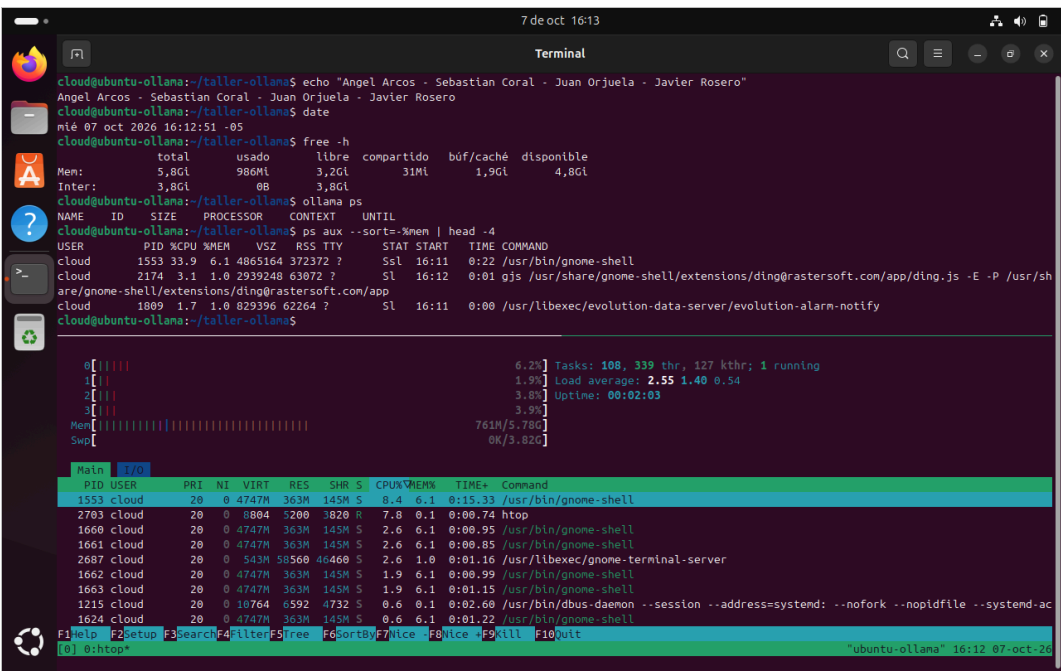


```
cloud@ubuntu-ollama: ~/taller-ollama
cloud@ubuntu-ollama:~/taller-ollama$ echo "Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero"
Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero
cloud@ubuntu-ollama:~/taller-ollama$ date
mié 07 oct 2026 16:12:39 -05
cloud@ubuntu-ollama:~/taller-ollama$ apt policy htop curl git
htop:
  Instalados: 3.3.0-4build1
  Candidato: 3.3.0-4build1
  Tabla de versión:
  *** 3.3.0-4build1 500
        500 http://archive.ubuntu.com/ubuntu noble/main amd64 Packages
        100 /var/lib/dpkg/status
curl:
  Instalados: 8.5.0-2ubuntu10.15
  Candidato: 8.5.0-2ubuntu10.15
  Tabla de versión:
  *** 8.5.0-2ubuntu10.15 500
        500 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages
        500 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages
        100 /var/lib/dpkg/status
        8.5.0-2ubuntu10 500
        500 http://archive.ubuntu.com/ubuntu noble/main amd64 Packages
git:
  Instalados: 1:2.43.0-1ubuntu7.3
  Candidato: 1:2.43.0-1ubuntu7.3
  Tabla de versión:
  *** 1:2.43.0-1ubuntu7.3 500
        500 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages
        500 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages
        100 /var/lib/dpkg/status
        1:2.43.0-1ubuntu7 500
        500 http://archive.ubuntu.com/ubuntu noble/main amd64 Packages
cloud@ubuntu-ollama:~/taller-ollama$ dpkg -l htop curl git | tail -3
ll curl      8.5.0-2ubuntu10.15 amd64      command line tool for transferring data with URL syntax
ll git       1:2.43.0-1ubuntu7.3 amd64      fast, scalable, distributed revision control system
ll htop      3.3.0-4build1      amd64      interactive processes viewer
cloud@ubuntu-ollama:~/taller-ollama$
```

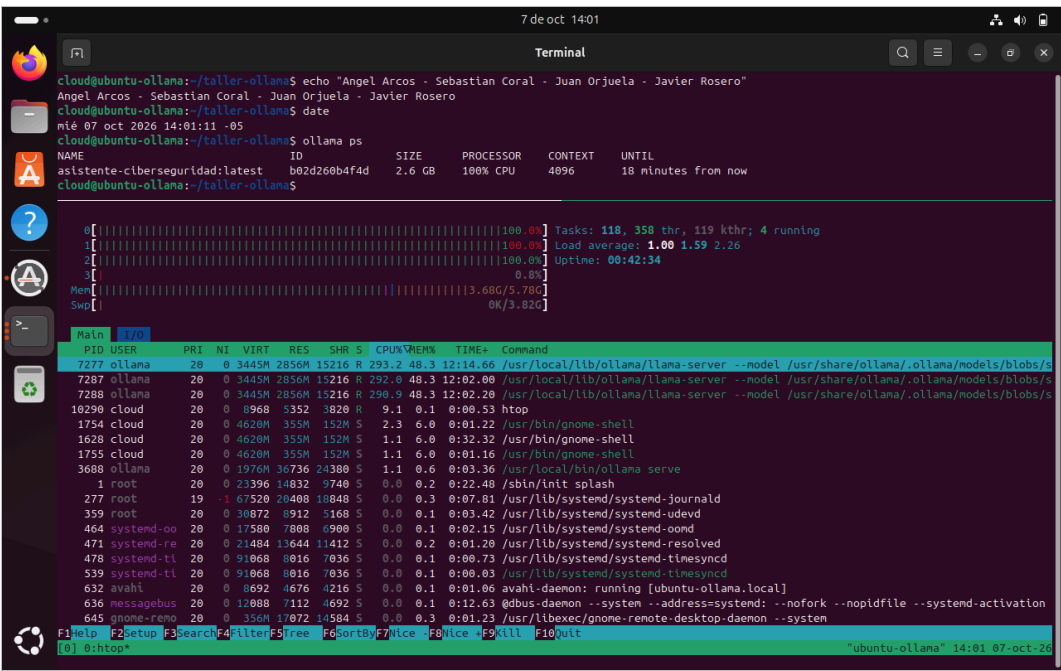
Captura 5. apt policy y dpkg -l confirman que htop, curl y git están instalados.

La diferencia que pide explicar: apt update solo descarga la lista de paquetes disponibles, o sea qué versiones hay en los repositorios, y no instala ni cambia nada. apt upgrade sí instala las versiones nuevas de lo que ya está instalado, usando esa lista. Por eso siempre va primero update. En apt policy se ve que de los tres paquetes la versión instalada es igual a la candidata, es decir, están al día.

Ejercicio 3: procesos antes, durante y después de la IA



Captura 6. Antes: sin modelos cargados (ollama ps vacío), cerca de 1 GB de RAM usado y las cuatro CPU casi quietas. El que más memoria usa es el escritorio (gnome-shell).



Captura 7. Durante una respuesta larga: las CPU 0, 1 y 2 al 100 % y la CPU 3 casi libre, por el AllowedCPUs=0-2 de la sección 3. Arriba en la lista aparece llama-server con 2,8 GB (48 % de la RAM).

```
cloud@ubuntu-ollama:~/taller-ollama$ echo "Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero"
Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero
cloud@ubuntu-ollama:~/taller-ollama$ date
mié 07 oct 2026 16:20:31 -05
cloud@ubuntu-ollama:~/taller-ollama$ ollama ps
NAME                                ID                                SIZE  PROCESSOR  CONTEXT  UNTIL
asistente-ciberseguridad:latest    b02d260b4f4d                    2.6 GB  100% CPU   4896     9 minutes from now
cloud@ubuntu-ollama:~/taller-ollama$ ollama stop asistente-ciberseguridad
cloud@ubuntu-ollama:~/taller-ollama$ ollama ps
NAME  ID  SIZE  PROCESSOR  CONTEXT  UNTIL
cloud@ubuntu-ollama:~/taller-ollama$ free -h
total        usado        libre    compartido    búf/caché    disponible
Mem:  5,8Gi    1,0Gi    2,6Gi    33Mi    2,5Gi    4,8Gi
Inter:  3,8Gi    324Ki    3,8Gi
cloud@ubuntu-ollama:~/taller-ollama$
```

```
0[||||] 4.0% Tasks: 109, 339 thr, 116 kthr; 2 running
1[|] 1.3% Load average: 1.17 0.99 0.67
2[|] 0.7% Uptime: 00:09:43
3[|] 0.0%
Mem[|||||] 785M/5.78G
Swp[|] 0K/3.82G
```

Main		I/O									
PID	USER	PRI	NI	VRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
2703	cloud	20	0	0876	5256	3020	R	5.3	0.1	0:18.81	htop
6094	cloud	20	0	0824	5180	3700	R	5.3	0.1	0:00.46	htop
1553	cloud	20	0	4749M	357M	140M	S	2.0	6.0	0:22.32	/usr/bin/gnome-shell
1660	cloud	20	0	4749M	357M	140M	S	0.7	6.0	0:02.29	/usr/bin/gnome-shell
1661	cloud	20	0	4749M	357M	140M	S	0.7	6.0	0:02.58	/usr/bin/gnome-shell
1662	cloud	20	0	4749M	357M	140M	S	0.7	6.0	0:02.71	/usr/bin/gnome-shell
1	root	20	0	23332	14632	9780	S	0.0	0.2	0:11.16	/sbin/init splash
271	root	19	-1	67116	19180	17672	S	0.0	0.3	0:03.70	/usr/lib/systemd/systemd-journald

F1 Help F2 Setup F3 Search F4 Filter F5 Free F6 Sort By F7 Nice F8 Nicer F9 Kill F10 Quit

[1] 0:htop*

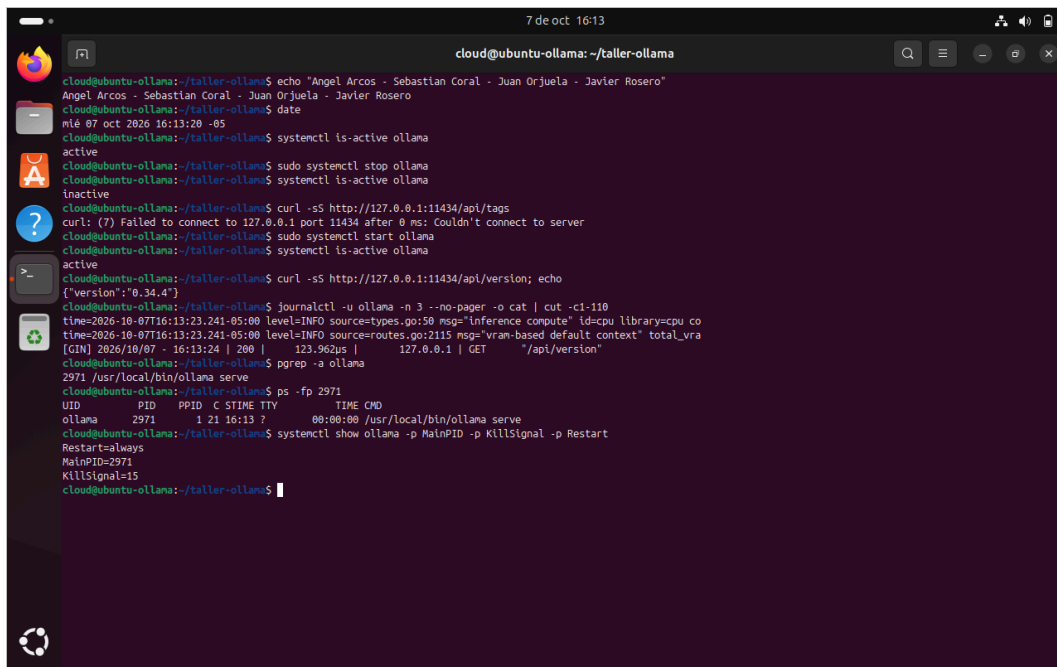
"ubuntu-ollama" 16:20 07-oct-26

Captura 8. Después: el modelo seguía cargado. Al hacer ollama stop se libera y la RAM usada vuelve a cerca de 1 GB.

Momento	RAM usada	RAM disponible	Qué se ve
Antes	~1,0 GiB	4,8 GiB	CPU casi en reposo; gnome-shell arriba
Durante	3,5 GiB	2,3 GiB	11ama-server con 2,5 a 2,8 GB y tres CPU al 100 %
Después, con el modelo aún cargado	3,5 GiB	2,3 GiB	La CPU baja, pero la memoria sigue ocupada
Después de ollama stop	1,0 GiB	4,8 GiB	Vuelve a como estaba antes

Conclusión: quien consume los recursos no es ollama serve (ocupa unos 36 MB) sino su proceso hijo 11ama-server, que es el que carga el modelo en memoria. Cuando termina de responder, Ollama deja el modelo cargado 5 minutos para que la siguiente pregunta no tenga que esperar la carga. Por eso la memoria no baja sola enseguida.

Ejercicios 4 y 5: el servicio, los procesos y las señales



```
cloud@ubuntu-ollama: ~/taller-ollama
cloud@ubuntu-ollama:~/taller-ollama$ echo "Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero"
Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero
cloud@ubuntu-ollama:~/taller-ollama$ date
mié 07 oct 2026 16:13:28 -05
cloud@ubuntu-ollama:~/taller-ollama$ systemctl is-active ollama
active
cloud@ubuntu-ollama:~/taller-ollama$ sudo systemctl stop ollama
cloud@ubuntu-ollama:~/taller-ollama$ systemctl is-active ollama
inactive
cloud@ubuntu-ollama:~/taller-ollama$ curl -s http://127.0.0.1:11434/api/tags
curl: (7) Failed to connect to 127.0.0.1 port 11434 after 0 ms: Couldn't connect to server
cloud@ubuntu-ollama:~/taller-ollama$ sudo systemctl start ollama
cloud@ubuntu-ollama:~/taller-ollama$ systemctl is-active ollama
active
cloud@ubuntu-ollama:~/taller-ollama$ curl -s http://127.0.0.1:11434/api/version; echo
{"version":"0.34.4"}
cloud@ubuntu-ollama:~/taller-ollama$ journalctl -u ollama -n 3 --no-pager -o cat | cut -c1-110
time=2026-10-07T16:13:23.241-05:00 level=INFO source=types.go:50 msg="inference compute" id=cpu library=cpu co
time=2026-10-07T16:13:23.241-05:00 level=INFO source=routes.go:2115 msg="vram-based default context" total_vra
[GIN] 2026/10/07 - 16:13:24 | 200 | 123.962us | 127.0.0.1 | GET | "/api/version"
cloud@ubuntu-ollama:~/taller-ollama$ pgrep -a ollama
2971 /usr/local/bin/ollama serve
cloud@ubuntu-ollama:~/taller-ollama$ ps -fp 2971
UID          PID    PPID  C STIME TTY          TIME CMD
ollama      2971        1  1 16:13 ?        00:00:00 /usr/local/bin/ollama serve
cloud@ubuntu-ollama:~/taller-ollama$ systemctl show ollama -p MainPID -p KillSignal -p Restart
Restart=always
MainPID=2971
KillSignal=15
cloud@ubuntu-ollama:~/taller-ollama$
```

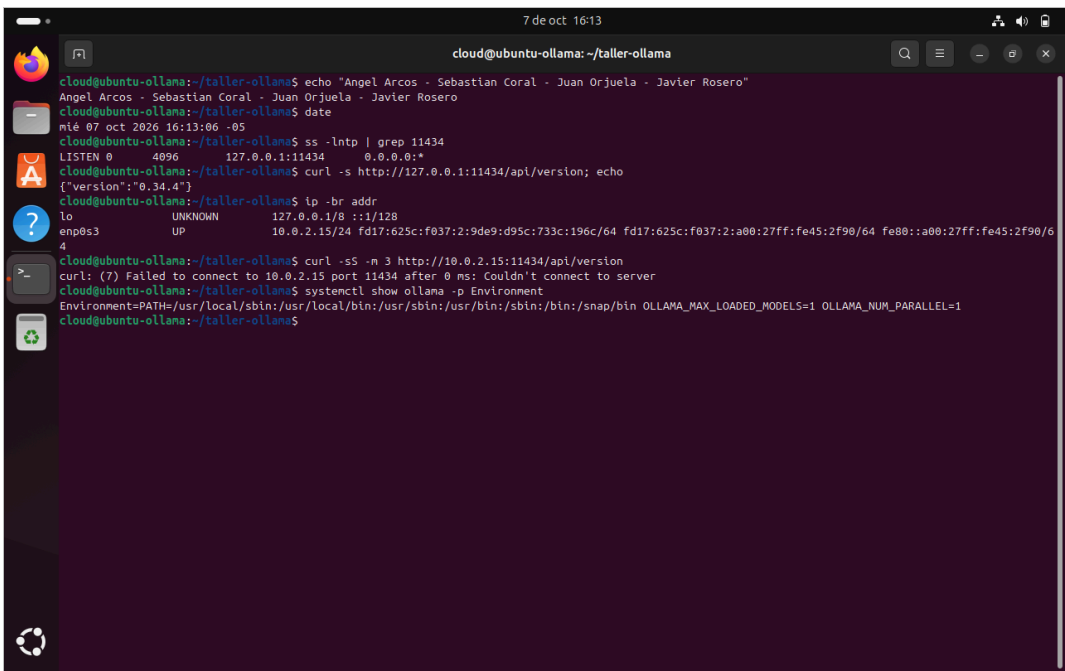
Captura 9. Se detiene el servicio, la API deja de responder, se inicia de nuevo y vuelve a responder. Abajo, los últimos logs, el PID con pgrep, el proceso con ps -fp y la configuración de systemd.

Ejercicio 4. Con `systemctl stop` el servicio pasa a inactive y `curl` falla con *Couldn't connect to server*. Con `systemctl start` vuelve a active y la API responde otra vez. `systemd` es el primer proceso que arranca Ubuntu (PID 1) y se encarga de los servicios: los inicia al prender la máquina (por eso aparece enabled), los vigila y los reinicia si se caen (`Restart=always`), mete todos sus procesos en un mismo grupo y guarda sus logs, que se consultan con `journalctl`.

Ejercicio 5. `pgrep -a ollama` nos dio el PID del servicio (2971 en esa corrida). Con `ps -fp` se ve que el dueño es el usuario `ollama` y que su padre (PPID) es el 1, o sea `systemd`. Para dejar claros los conceptos:

- **Proceso:** un programa en ejecución. **PID:** el número con el que el sistema lo identifica. **Servicio:** un proceso que corre en segundo plano y que administra `systemd`.
- **Señal:** un aviso que el sistema le manda a un proceso. **SIGTERM (15)** le pide que termine, y el proceso puede cerrar sus cosas con calma antes de salir. Es la que usa `systemd` (`KillSignal=15`). **SIGKILL (9)** lo mata de una, sin darle chance de nada, y puede dejar archivos a medias. No la usamos sobre el servicio, como pide la guía.
- Un servicio hay que manejarlo con `systemctl` y no con `kill`: como `systemd` lo vigila y tiene `Restart=always`, si lo matamos a mano lo vuelve a levantar a los 3 segundos. Además `systemctl stop` detiene también los procesos hijos, como `ollama-server`.

Ejercicio 6: red y puerto de Ollama



Captura 10. El socket escucha en 127.0.0.1:11434. La API responde por esa dirección, pero no por la IP de la VM (10.0.2.15), y no hay ninguna variable `OLLAMA_HOST` configurada.

Ollama escucha solo en **127.0.0.1**, que es la dirección interna de la propia máquina (loopback). Eso quiere decir que solo los programas que corren dentro de la VM pueden usarlo. Ni siquiera responde si le hablamos por la IP de la VM. Si escuchara en `0.0.0.0`, cualquier equipo que llegue a la VM por la red podría usar el modelo, y la API no pide usuario ni contraseña. Por eso no lo cambiamos ni abrimos ese puerto en el NAT.

Ejercicio 7: almacenamiento de modelos

Cuando bajamos los tres modelos principales anotamos el disco antes y después:

Modelo	Tamaño	Disco usado antes y después	Tiempo de descarga
gemma3:1b	815 MB	de 15 G (39 %) a 16 G (41 %)	169 s
llama3.2:3b	2,0 GB	de 16 G (41 %) a 18 G (46 %)	432 s
qwen2.5:1.5b	986 MB	de 17 G (46 %) a 18 G (48 %)	63 s

Para que se viera en una captura repetimos la prueba con un modelo muy pequeño (`smollm2:135m`) y medimos en megas:

```
cloud@ubuntu-ollama: ~/taller-ollama
cloud@ubuntu-ollama:~/taller-ollama$ echo "Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero"
Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero
cloud@ubuntu-ollama:~/taller-ollama$ date
mié 07 oct 2026 16:21:57 -05
cloud@ubuntu-ollama:~/taller-ollama$ df -BM / | tail -1
/dev/sda2      40140M 18761M    19470M   50% /
cloud@ubuntu-ollama:~/taller-ollama$ ollama list
NAME                                ID                                SIZE  MODIFIED
asistente-temp10:latest             69106e430c5c                     2.0 GB  3 hours ago
asistente-temp07:latest             0b37b0e25653                     2.0 GB  3 hours ago
asistente-temp01:latest             1f5292ec78ef                     2.0 GB  3 hours ago
asistente-ciberseguridad:latest     b02d260b4f4d                     2.0 GB  3 hours ago
qwen2.5:1.5b                        65ec06548149                     906 MB  12 days ago
llama3.2:3b                         a80c4f17acd5                     2.0 GB  12 days ago
gemma3:1b                           8648f39daa8f                     815 MB  12 days ago
cloud@ubuntu-ollama:~/taller-ollama$ ollama pull smollm2:135m 2>&1 | tail -1
success
cloud@ubuntu-ollama:~/taller-ollama$ df -BM / | tail -1
/dev/sda2      40140M 19019M    19211M   50% /
cloud@ubuntu-ollama:~/taller-ollama$ ollama cp smollm2:135m prueba-alias
copied 'smollm2:135m' to 'prueba-alias'
cloud@ubuntu-ollama:~/taller-ollama$ ollama list | grep -E "NAME|smollm|prueba"
NAME                                ID                                SIZE  MODIFIED
prueba-alias:latest                 9077fe9d2ae1                     270 MB  Less than a second ago
smollm2:135m                        9077fe9d2ae1                     270 MB  Less than a second ago
cloud@ubuntu-ollama:~/taller-ollama$ ollama rm prueba-alias smollm2:135m
deleted 'prueba-alias'
deleted 'smollm2:135m'
cloud@ubuntu-ollama:~/taller-ollama$ df -BM / | tail -1
/dev/sda2      40140M 18761M    19470M   50% /
cloud@ubuntu-ollama:~/taller-ollama$
```

Captura 11. Antes: 18 761 MB usados. Después de ollama pull: 19 019 MB (258 MB más). ollama cp crea un alias con el mismo ID, y después de ollama rm el disco vuelve a 18 761 MB.

Los modelos se guardan en `/usr/share/ollama/.ollama/models`, que es la carpeta del usuario ollama. Nuestro usuario normal no puede entrar (*Permiso denegado*). Adentro hay dos carpetas: `blobs/`, con los archivos grandes nombrados por su hash, y `manifests/`, con la lista de qué archivos forman cada modelo. Por eso asistente-ciberseguridad aparece con 2,0 GB en `ollama list` pero no ocupa espacio extra: usa el mismo archivo de llama3.2:3b. En total los modelos ocupan unos 3,6 GB de los 40 GB. También intentamos bajar qwen2.5:7b (4,7 GB), pero la descarga se cortó con los congelamientos y dejó unos 2 GB de archivos a medias (`*-partial`), que borramos a mano.

Ejercicio 8: rendimiento y selección del modelo

A los tres modelos les hicimos la misma pregunta por la API, con la misma configuración (temperatura 0.2, semilla 42, 3 hilos) y descargando cada uno de la memoria antes de probar el siguiente: *"Explica en máximo 120 palabras qué es un proceso en un sistema operativo y en qué se diferencia de un hilo. Da un ejemplo."*

Modelo	Tamaño	RAM	Tiempo		¿Funciona?	Calidad de la respuesta
			total	Velocidad		
gemma3:1b	815 MB	900 MB	39,0 s	6,4 tok/s	Sí	La mejor en esta pregunta: define los dos conceptos, marca la diferencia y pone de ejemplo un navegador
qwen2.5:1.5b	986 MB	1 131 MB	32,0 s	7,1 tok/s	Sí	Correcta pero muy básica; el ejemplo no ayuda mucho
llama3.2:3b	2,0 GB	2 470 MB	62,3 s	3,9 tok/s	Sí	Tiene un error ("un proceso se ejecuta en el núcleo"), pero el ejemplo del editor con hilos es bueno
qwen2.5:7b	4,7 GB				No	Cargado ocuparía unos 5,5 GB de los 6 GB de la VM; la descarga ni siquiera terminó

Algo que no esperábamos: la velocidad cambió muchísimo según el día. Como usamos semilla fija, las respuestas salieron idénticas palabra por palabra, pero el 24 de septiembre la misma prueba dio 22,6 / 24,4 / 11,6 tok/s y hoy dio 6,4 / 7,1 / 3,9. La diferencia es lo que esté haciendo Windows al mismo tiempo, por el modo lento de la sección 3. Lo que sí se mantuvo siempre es el orden: los modelos de 1 y 1,5B son más o menos el doble de rápidos que el de 3B.

Escogimos **llama3.2:3b** para el asistente. Es el más lento, pero fue el que mejor siguió en español la estructura de cuatro partes del Modelfile, y sus 2,5 GB caben bien en los 6 GB de la VM. Siendo honestos, en esta pregunta gemma3:1b respondió mejor y es más rápido. Con una sola pregunta no se puede decidir cuál es mejor en general; lo siguiente sería probar el mismo Modelfile con gemma3:1b y varias preguntas.

Ejercicio 9: automatización con Bash

```

7 de oct 16:14
cloud@ubuntu-ollama: ~/taller-ollama
cloud@ubuntu-ollama:~/taller-ollama$ echo "Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero"
Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero
cloud@ubuntu-ollama:~/taller-ollama$ date
mié 07 oct 2026 16:14:01 -05
cloud@ubuntu-ollama:~/taller-ollama$ cat reporte_sistema.sh
#!/bin/bash
echo "=== REPORTE DEL SISTEMA ==="
date
hostname
uname -a
echo "--- CPU ---"
nproc
LC_ALL=C lscpu | grep "Model name" # LC_ALL=C: con Ubuntu en español lscpu imprime "Nombre del modelo"
echo "--- RAM ---"
free -h
echo "--- DISCO ---"
df -h /
echo "--- OLLAMA ---"
systemctl is-active ollama
cloud@ubuntu-ollama:~/taller-ollama$ chmod +x reporte_sistema.sh
cloud@ubuntu-ollama:~/taller-ollama$ ./reporte_sistema.sh | tee reporte.txt
=== REPORTE DEL SISTEMA ===
mié 07 oct 2026 16:14:01 -05
ubuntu-ollama
Linux ubuntu-ollama 7.0-0-34-generic #34-24.04.1-Ubuntu SMP PREEMPT_DYNAMIC Fri Sep 4 15:38:29 UTC 2 x86_64 x86_64 x86_64 GNU/Linux
--- CPU ---
4
Model name:
12th Gen Intel(R) Core(TM) i9-12900H
--- RAM ---
total usado libre compartido buff/cache disponible
Mem: 5,8Gi 1,0Gi 3,2Gi 34Mi 1,9Gi 4,8Gi
Inter: 3,8Gi 0B 3,8Gi
--- DISCO ---
S.ficheros Tamaño Usados Disp Usd% Montado en
/dev/sda2 49G 19G 28G 58% /
--- OLLAMA ---
active
cloud@ubuntu-ollama:~/taller-ollama$ ls -l reporte.txt
-rw-rw-r-- 1 cloud cloud 648 oct 7 16:14 reporte.txt
cloud@ubuntu-ollama:~/taller-ollama$

```

Captura 12. El script, el permiso de ejecución, la ejecución guardando la salida con tee reporte.txt y el archivo creado.

Copiamos el script de la guía, pero la línea de la CPU no imprimía nada. El problema era que nuestro Ubuntu está en español y `lscpu` escribe "Nombre del modelo" en vez de "Model name", así que el `grep` no encontraba nada. Lo arreglamos con `LC_ALL=C lscpu | grep "Model name"`, que hace que solo ese comando salga en inglés. Después le dimos permiso con `chmod +x` y lo corrimos con `./reporte_sistema.sh | tee reporte.txt`. Los dos archivos están en la carpeta `codigo/`.

Ejercicio 10: la IA como apoyo, no como administrador

Le pasamos al modelo la salida de `free -h`, `df -h` /, las primeras líneas de `systemctl status ollama` y `ps aux --sort=-%mem | head -6` (nada sensible) y le pedimos que nos dijera cuánta RAM había disponible, si el disco era un riesgo, si el servicio estaba bien y qué procesos usaban más memoria. Después revisamos cada cosa que dijo:

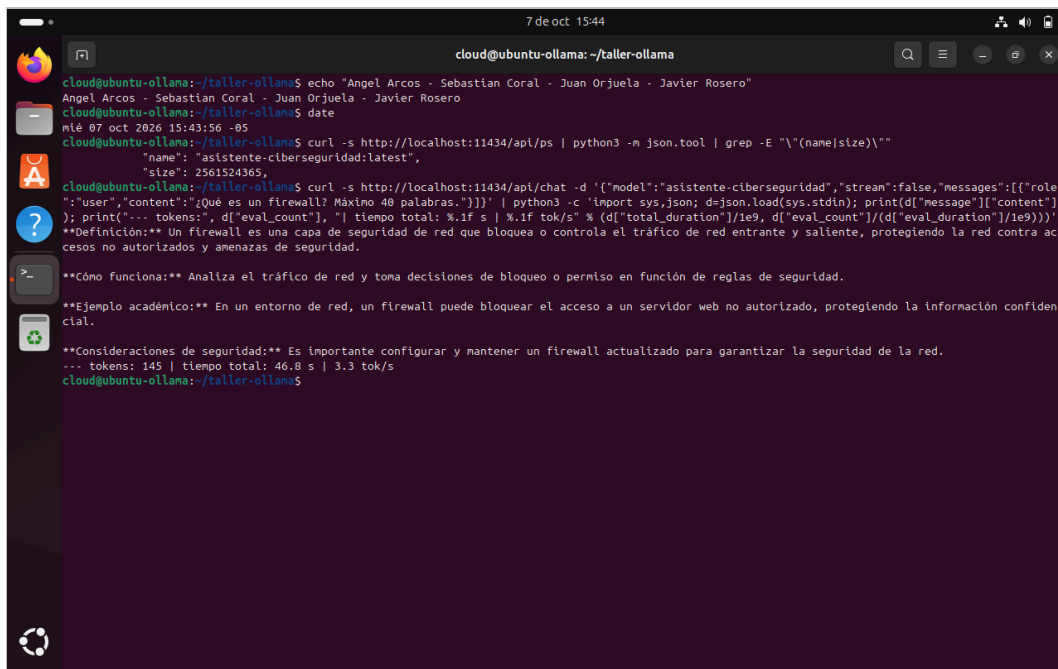
Lo que dijo la IA	Lo que vimos en Ubuntu	Resultado
"Hay 2,3 GiB disponibles; la compartida y el caché también suman a la disponible"	El 2,3 GiB es correcto. Lo del caché está a medias: la columna <i>disponible</i> ya incluye el caché que se puede liberar, no hay que sumarlo aparte	Parcial
"Hay 21 GiB libres; el disco no es un riesgo"	<code>df</code> dice 20 G libres. El modelo restó 40 – 19 por su cuenta en vez de leer el dato. La conclusión sí está bien	Parcial
"El proceso principal de ollama usa 3,8 GiB y un uso de CPU de 1h 2min"	Los 3,8 G son de todo el grupo del servicio, incluido el modelo; el proceso principal usa unos 36 MB. Y "1h 2min" es el tiempo total de CPU acumulado, no un porcentaje	Incorrecto
Lista "Ollama (PID 7277)" y "Llama-server (PID 7277)" como dos procesos	Es uno solo, llama-server. Repitió la fila	Incorrecto
"El servicio está sano y no consume demasiada memoria"	Sano sí (active y 0 reinicios), pero llama-server usa el 42,5 % de la RAM, así que lo de "no demasiada" no se sostiene	Parcial

Conclusión: el modelo lee bien los números, pero a veces hace cuentas en vez de leer el dato, confunde conceptos y repite cosas. En una primera prueba también inventó una conversión de GiB a bytes. Sirve para orientarse, pero no para administrar el sistema sin revisar.

7. API REST (sección 08)

Probamos `/api/generate` y `/api/chat` con `curl` desde la VM:

Llamada	Tiempo	Tokens	Velocidad	Observación
<code>/api/generate: "¿Qué es ciberseguridad?"</code>	106,2 s	238	3,1 tok/s	Incluye 14,4 s cargando el modelo
<code>/api/chat: "¿Qué es un firewall?"</code>	71,2 s	220	3,2 tok/s	El modelo ya estaba cargado
<code>/api/chat: segunda pregunta con historial</code>	58,8 s	155		Entran 368 tokens porque se reenvía toda la conversación

A terminal window titled 'cloud@ubuntu-ollama: ~/taller-ollama' with a timestamp of '7 de oct 15:44'. The terminal shows a series of commands and their outputs. The first command is 'echo "Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero"', which outputs the names. The second is 'date', showing the date and time. The third is a 'curl' command to 'http://localhost:11434/api/ps', which returns JSON data about the loaded model. The fourth is another 'curl' command to 'http://localhost:11434/api/chat' with a specific payload, which returns a detailed response about firewalls, including a definition, how it works, an academic example, and security considerations, along with token and time statistics.

```
cloud@ubuntu-ollama:~/taller-ollama$ echo "Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero"
Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero
cloud@ubuntu-ollama:~/taller-ollama$ date
mié 07 oct 2026 15:43:56 -05
cloud@ubuntu-ollama:~/taller-ollama$ curl -s http://localhost:11434/api/ps | python3 -m json.tool | grep -E "(name|size)\\""
  "name": "asistente-ciberseguridad:latest",
  "size": 2561524365,
cloud@ubuntu-ollama:~/taller-ollama$ curl -s http://localhost:11434/api/chat -d '{"model": "asistente-ciberseguridad", "stream": false, "messages": [{"role": "user", "content": "¿Qué es un firewall? Máximo 40 palabras."}]]}' | python3 -c 'import sys,json; d=json.load(sys.stdin); print(d["message"]["content"]); print("--- tokens:", d["eval_count"], "| tiempo total: %.1f s | %.1f tok/s" % (d["total_duration"]/1e9, d["eval_count"]/(d["eval_duration"]/1e9)))'
**Definición:** Un firewall es una capa de seguridad de red que bloquea o controla el tráfico de red entrante y saliente, protegiendo la red contra accesos no autorizados y amenazas de seguridad.

**Cómo funciona:** Analiza el tráfico de red y toma decisiones de bloqueo o permiso en función de reglas de seguridad.

**Ejemplo académico:** En un entorno de red, un firewall puede bloquear el acceso a un servidor web no autorizado, protegiendo la información confidencial.

**Consideraciones de seguridad:** Es importante configurar y mantener un firewall actualizado para garantizar la seguridad de la red.
--- tokens: 145 | tiempo total: 46.8 s | 3.3 tok/s
cloud@ubuntu-ollama:~/taller-ollama$
```

Captura 13. /api/ps muestra el modelo cargado, y una consulta a /api/chat con curl devuelve la respuesta, los tokens y el tiempo.

Aprendimos que la API no recuerda nada entre consultas. El "historial" funciona porque el cliente vuelve a mandar todos los mensajes anteriores, y por eso cada pregunta nueva pesa más.

Ejemplo de PowerShell desde Windows. La guía también trae la llamada con Invoke-RestMethod, que se corre en el anfitrión. El problema es que Ollama solo escucha dentro de la VM (ejercicio 6), y no queríamos abrir ese puerto. La solución fue un túnel SSH: con `ssh -L 11434:127.0.0.1:11434 -p 2222 cloud@127.0.0.1`, el puerto 11434 de Windows queda conectado al de la VM a través de la conexión SSH, que ya es cifrada, sin publicar nada en la red. Con el túnel abierto, esto es lo que salió en PowerShell (en Windows PowerShell 5.1 hubo que agregar `charset=utf-8` para que las tildes de la pregunta llegaran bien):

```

PS C:\Users\juanp> echo "Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero"
Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero
PS C:\Users\juanp> Get-Date
Wednesday, October 7, 2026 5:02:32 PM
PS C:\Users\juanp> (Invoke-RestMethod -Uri "http://localhost:11434/api/tags").models.name
asistente-temp10:latest
asistente-temp07:latest
asistente-temp01:latest
asistente-ciberseguridad:latest
qwen2.5:1.5b
llama3.2:3b
gemma3:1b
PS C:\Users\juanp> $body = @{ model = "asistente-ciberseguridad"; prompt = "¿Qué es ciberseguridad? Máximo
40 palabras."; stream = $false } | ConvertTo-Json
PS C:\Users\juanp> $r = Invoke-RestMethod -Uri "http://localhost:11434/api/generate" -Method Post -
ContentType "application/json; charset=utf-8" -Body $body
PS C:\Users\juanp> $r.response
**Definición:** La ciberseguridad es el conjunto de medidas y técnicas para proteger la información,
sistemas y redes de las amenazas y vulnerabilidades en el ámbito digital.
**Cómo funciona:** Implica la identificación y mitigación de riesgos, la implementación de controles de
seguridad y la educación de los usuarios para prevenir ataques cibernéticos.
**Ejemplo académico:** En un estudio sobre ciberseguridad, se analizaron los ataques a la base de datos de
una empresa de salud, y se implementaron medidas de seguridad para mejorar la protección de la
información.
**Consideraciones de seguridad:** La ciberseguridad requiere una atención constante para mantenerse
actualizado con las últimas amenazas y vulnerabilidades, y para garantizar la protección de la información
y los sistemas.
PS C:\Users\juanp> "tokens: {0} | tiempo total: {1:N1} s" -f $r.eval_count, ($r.total_duration / 1e9)
tokens: 184 | tiempo total: 57.7 s

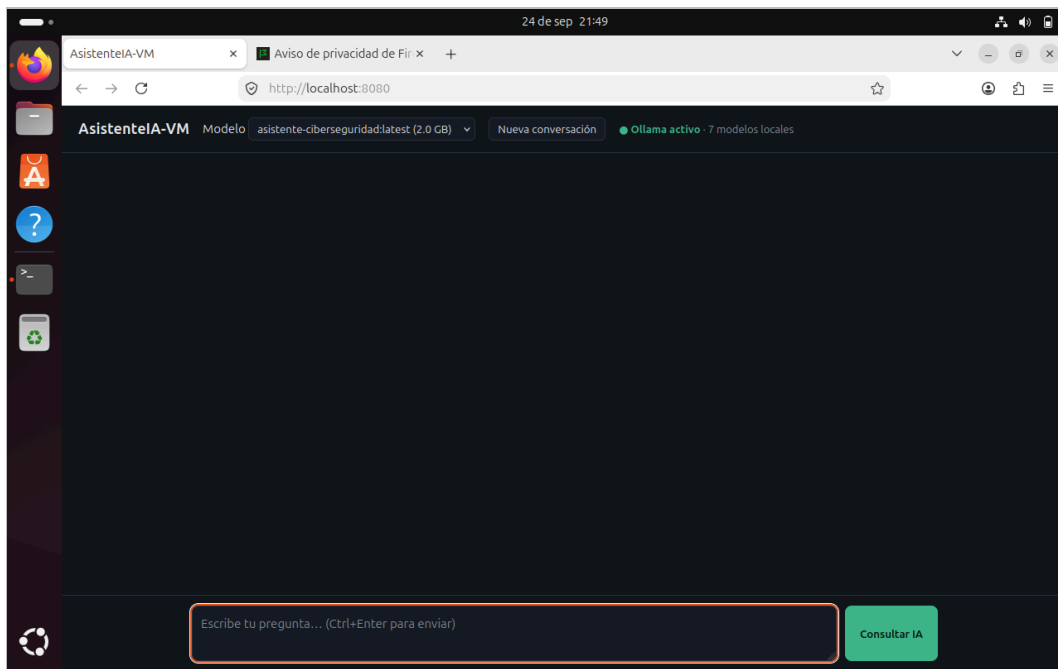
```

Otra vez el "ejemplo académico" es genérico y no cita nada verificable, igual que en la sección 5.

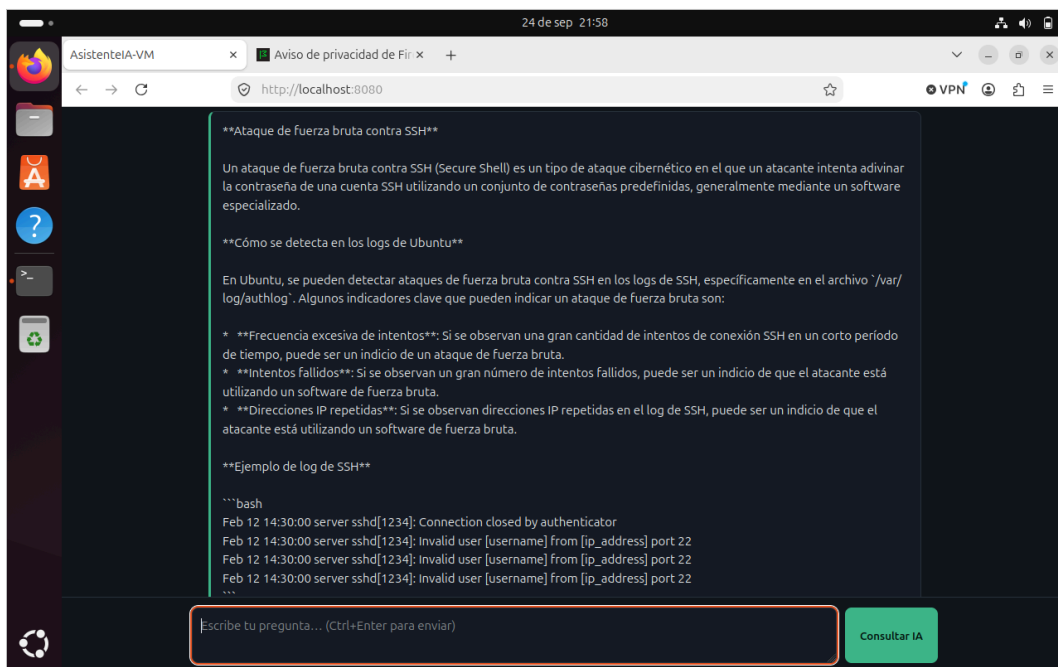
8. Interfaz HTML + JavaScript: AsistentelA-VM (secciones 09 y 13)

Partimos del ejemplo de la guía y le fuimos agregando cosas. La página se sirve con `servir.sh`, que levanta un servidor local en el puerto 8080, en vez de abrirla con `file:///`. Así el navegador no bloquea las peticiones a Ollama y no tuvimos que tocar CORS. La aplicación tiene:

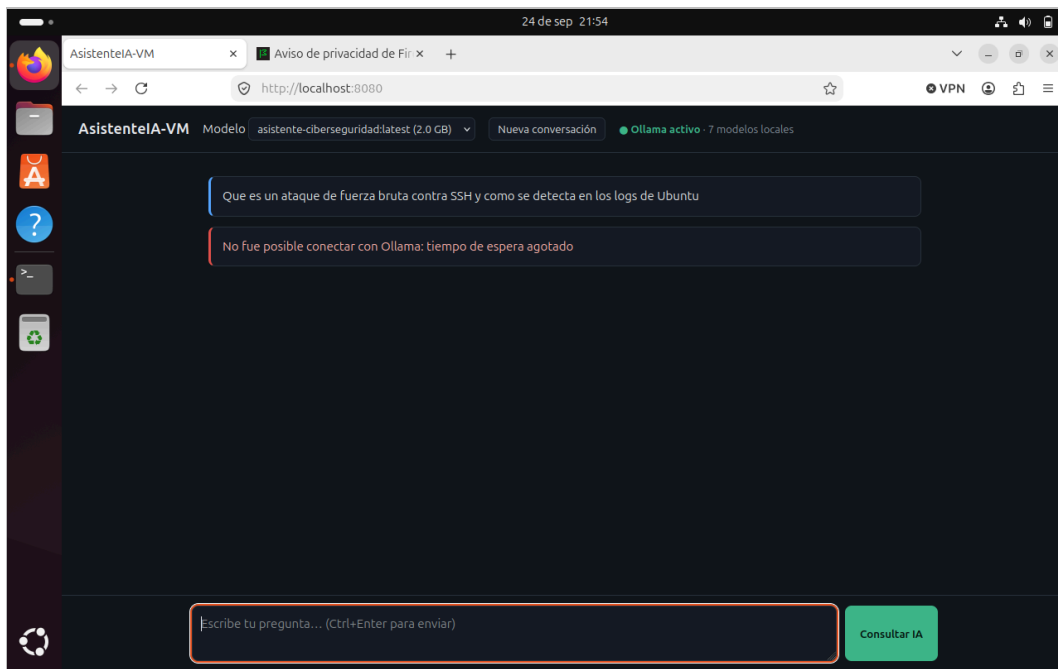
- Historial de conversación con `/api/chat` y el prompt de sistema de ciberseguridad, más un botón para empezar de nuevo.
- Un selector con los modelos instalados (sale de `/api/tags`) y un indicador de si Ollama está activo.
- Tiempo, tokens y velocidad de cada respuesta.
- Respuesta en tiempo real (*streaming*) y mensajes de error claros si Ollama no responde.



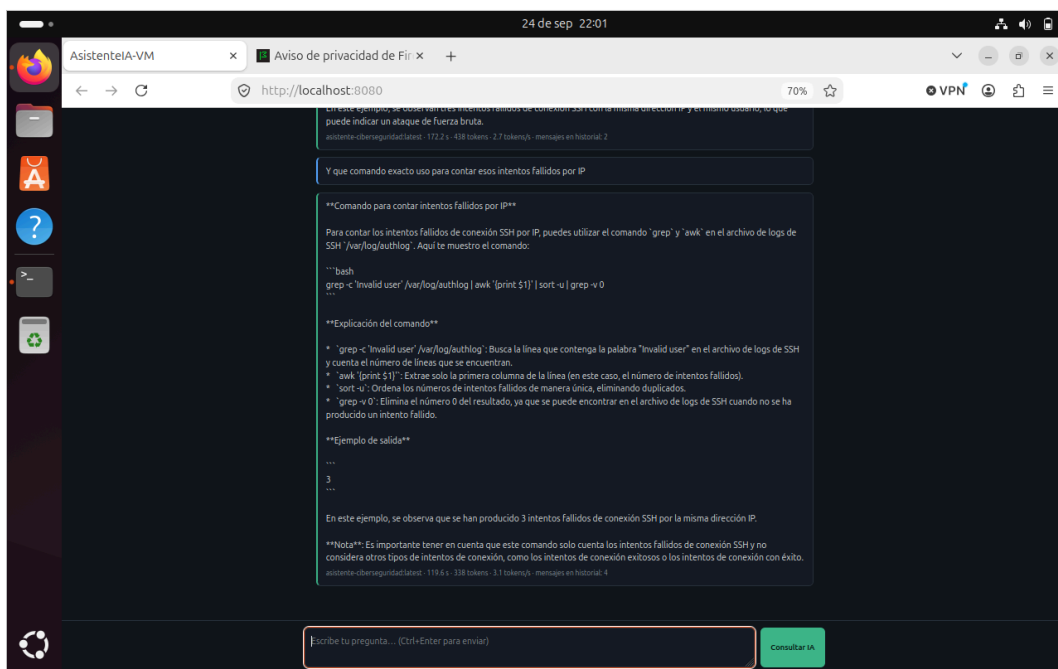
Captura 14. Pantalla inicial en `http://localhost:8080` dentro de la VM.



Captura 15. Primera respuesta, sobre ataques de fuerza bruta contra SSH. Ya aparece la ruta equivocada `/var/log/authlog`, que revisamos más abajo.



Captura 16. La primera versión esperaba la respuesta completa con un límite de 5 minutos. Con Firefox abierto la VM iba tan lenta (2,3 tok/s) que se pasó del tiempo. El mensaje de error funcionó, pero quedarse 5 minutos sin ver nada era muy incómodo, y por eso cambiamos a streaming.



Captura 17. Versión final con streaming. La segunda pregunta ("Y qué comando exacto uso para contar esos intentos fallidos por IP") solo tiene sentido con la primera, y el modelo la entendió gracias al historial.

Revisamos el comando que nos dio el asistente (`grep -c 'Invalid user' /var/log/authlog | awk ... | sort -u`) y está mal por dos lados. El archivo se llama `/var/log/auth.log`, no `authlog` (al hacer `ls` sale *No existe el archivo*). Y `grep -c` devuelve un solo número total, así que no se puede separar por IP. Un comando que sí funciona es:

```
sudo journalctl -u ssh | grep -E "Failed password|Invalid user" | grep -oE "from [0-9.]+ " | sort | uniq -c | sort -rn
```

En nuestra VM no devolvió nada, porque nunca hubo intentos fallidos: siempre entramos con llave SSH.

9. Ciberseguridad y reto de red (secciones 10 y 12)

1. Captura. Para no capturar tráfico de nadie más, capturamos solo el SSH entre nuestro portátil y nuestra VM. Usamos tshark, que es Wireshark en terminal: `sudo tshark -i enp0s3 -f 'tcp port 22' -a duration:25`, mientras abríamos sesiones SSH desde Windows.

```
cloud@ubuntu-ollama: ~/taller-ollama
Angel Arcos - Sebastian Coral - Juan Orjuela - Javier Rosero
cloud@ubuntu-ollama:~/taller-ollama$ date
mié 07 oct 2026 16:22:41 -05
cloud@ubuntu-ollama:~/taller-ollama$ capinfos -c -d -u evidencias/captura_ssh.pcapng | tail -3
Number of packets: 1338
Data size: 882 kB
Capture duration: 22,483804605 seconds
cloud@ubuntu-ollama:~/taller-ollama$ tshark -r evidencias/captura_ssh.pcapng -q -z conv,tcp | sed -n 5,12p | cut -c1-118
| Frames Bytes | | Frames Bytes | | Frames Bytes |
181 639 kB | 632 51 kB | 813 690 kB | 14,
105 39 kB | 149 18 kB | 254 57 kB | 19,
31 7.261 bytes | 40 6.622 bytes | 71 13 kB
32 7.499 bytes | 39 6.592 bytes | 71 14 kB
31 7.917 bytes | 39 6.592 bytes | 70 14 kB
26 6.087 bytes | 33 5.792 bytes | 59 11 kB
=====
cloud@ubuntu-ollama:~/taller-ollama$ tshark -r evidencias/captura_ssh.pcapng -Y ssh.protocol -T fields -e ip.src -e ssh.protocol | head -2
10.0.2.2 SSH-2,0-OpenSSH_10.3
10.0.2.15 SSH-2,0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.19
cloud@ubuntu-ollama:~/taller-ollama$ tshark -r evidencias/captura_ssh.pcapng -Y ssh.message_code -T fields -e frame.number -e ip.src -e ssh.message_code | head -5
9 10.0.2.2 20
12 10.0.2.15 20
14 10.0.2.2 30
16 10.0.2.15 31,21
19 10.0.2.2 21
cloud@ubuntu-ollama:~/taller-ollama$
```

Captura 18. Resumen de la captura con capinfos, las conversaciones TCP, los banners de SSH y los primeros mensajes de negociación.

La guía permite usar una captura de SSH o de una VPN. Escogimos SSH porque es el tráfico que ya teníamos entre el portátil y la VM, sin depender de un servidor externo. En una VPN (por ejemplo WireGuard u OpenVPN) se verían el mismo tipo de metadatos: las IP de los dos extremos del túnel, el protocolo y el puerto (normalmente UDP), el volumen y los tiempos. Lo que va dentro del túnel quedaría cifrado igual que en SSH.

2. Extracción. Sacamos solo los metadatos: 1 338 paquetes, 802 kB y 22,5 segundos. Hubo seis conexiones distintas desde 10.0.2.2 (así se ve el portátil a través del NAT de VirtualBox) hacia el puerto 22 de la VM (10.0.2.15): una sesión de 7,7 s, una copia de archivos con scp de 690 kB y cuatro conexiones cortas de comandos sueltos. En cada conexión se ve el intercambio de versiones (cliente OpenSSH_10.3 de Windows y servidor OpenSSH_9.6p1 Ubuntu) y la negociación de claves: mensajes 20, 30, 31 y 21. Después del mensaje 21 todo va cifrado.

3. Interpretación con IA. Le pasamos esos datos al asistente con el prompt de la guía, pidiéndole que separara lo observable, lo que se puede inferir, lo que sigue protegido por el cifrado y lo que habría que

volver a revisar en Wireshark.

4. Validación. Revisamos su respuesta contra la captura:

Lo que dijo el modelo	Lo que muestra la captura	Resultado
Protocolo SSH, puerto 22, IP de origen y destino, 1 338 paquetes, 802 360 bytes	Coincide con capinfos	Correcto
Habla de "la conexión", como si fuera una sola	Hay seis conexiones, y la del scp tiene el 86 % de los bytes	Incompleto
"Los banners muestran la versión de OpenSSH de la máquina Ubuntu"	Solo uno es de Ubuntu; OpenSSH_10.3 es el cliente de Windows. No los diferencia	Incompleto
"El contenido no se puede leer sin la contraseña o la clave privada"	No es así. SSH cifra con claves de sesión que se negocian en cada conexión (mensajes 30 y 31). Aunque alguien tuviera la clave privada, no podría descifrar lo capturado. Además, no usamos contraseña sino llave	Incorrecto
"Tener el puerto 22 abierto podría indicar una mala configuración de seguridad"	Eso no se puede saber con la captura. El puerto solo está reenviado dentro del portátil	Incorrecto

El modelo ordena bien la respuesta, pero se equivoca justo en la parte de seguridad (cómo cifra SSH) y se le escapan detalles que solo se ven volviendo a la captura.

10. Seguridad y operación (sección 11)

Punto del checklist	Cómo lo cumplimos
El modelo se ejecuta en un equipo controlado	Una VM propia, con red NAT y snapshots
No se exponen credenciales o secretos	Al modelo solo le pasamos datos del sistema y metadatos de red; al SSH entramos con llave
Las capturas de red fueron autorizadas	Solo tráfico entre nuestro portátil y nuestra VM
Se validaron las respuestas de la IA	En el Modelfile, el ejercicio 10, la interfaz y el reto. En todos encontramos errores
Se documentó el origen de los datos	Cada captura muestra los comandos y empieza con el echo del grupo y la fecha
Se evaluó el riesgo de exposición de la API	Los puertos 11434 y 8080 solo escuchan en 127.0.0.1, no se abrieron en el NAT y no usamos OLLAMA_HOST=0.0.0.0

Limitaciones: la VM no tiene GPU, la velocidad depende mucho de lo que esté haciendo Windows y no caben modelos de 7B con 6 GB. Además, la API de Ollama no tiene usuarios ni contraseña, así que para

usarla en serio habría que ponerle delante un servidor con control de acceso. Y como dejamos un solo modelo cargado a la vez, cambiar de modelo obliga a esperar que se cargue de nuevo.

11. Proyecto final: qué cumple AsistenteIA-VM (sección 13)

Requisito	Dónde está
Ollama como servicio	Secciones 2 y 3, y ejercicio 4
Modelo seleccionado y justificado	Ejercicio 8 (llama3.2:3b)
Modelfile	Sección 5 y código/AsistenteIA-VM/Modelfile
API REST e interfaz HTML/JavaScript	Secciones 7 (cURL y PowerShell) y 8, y código/AsistenteIA-VM/index.html
Consulta individual y manejo de errores	Capturas 14 a 16
Historial con /api/chat	Captura 17 y sección 7
Prompt de sistema de ciberseguridad	Sección 5
Prueba con datos de Wireshark	Sección 9
Tiempos y rendimiento	Ejercicio 8, sección 7 y métricas de la interfaz
Limitaciones y controles de seguridad	Sección 10
Servicio, procesos, recursos, almacenamiento y red	Ejercicios 3 a 7

12. Conclusiones

1. La IA nos sirvió como excusa para ver cómo Ubuntu maneja una carga pesada en todos sus recursos: tres CPU al 100 %, la memoria pasando de 1 GB a 3,5 GB, 3,6 GB de disco solo en modelos, un servicio en un puerto local y systemd administrándolo todo.
2. Lo que más aprendimos no estaba en la guía: la VM se congelaba porque VirtualBox corría encima de Hyper-V y el modelo ocupaba todos los núcleos, y además le habíamos dado demasiada RAM. Lo arreglamos con cosas vistas en el curso (cgroups con AllowedCPUs, configuración de systemd y menos hilos), probando un cambio a la vez.
3. En una VM sin GPU el tamaño del modelo es lo que más pesa. Los de 1 y 1,5B ocupan cerca de 1 GB y van al doble de velocidad que el de 3B. La velocidad exacta cambió mucho entre días, pero ese orden se mantuvo.
4. En todas las pruebas el modelo cometió errores que suenan creíbles: casos y citas inventados, un comando que no funciona, cuentas mal hechas y una explicación equivocada de cómo se cifra SSH. Es útil para explicar y organizar ideas, pero no se le puede dejar administrar el sistema sin revisar lo que dice.

Anexo: estructura del repositorio

```
tallerOllama1/  
├─ Informe_Taller_Ollama_VM.pdf  
├─ README.md  
└─ codigo/  
    ├─ AsistenteIA-VM/  index.html, Modelfile (y las 3 variantes de temperatura), servir.sh, README.md  
    ├─ reporte_sistema.sh (corregido), reporte_sistema_original.sh, reporte.txt  
    ├─ scripts/         un script por ejercicio, bench.py, temperaturas.py y los de las capturas  
    └─ lib.sh, rehacer.sh  (corren los scripts en orden con el echo del grupo)
```